Today

Welcome to your next lesson: Creating your first class.

I'm preparing your personalized lesson based on what you've already learned, what's coming up next, and the best way to strengthen your skills.

Step by step, you’re moving forward.

Ron, let’s turn your blueprint vs instance understanding into real Python code now 🐾

Imagine we want a blueprint for any pet. In Python, the minimal version of that blueprint looks like this:

Python
```
class Pet:
    pass
```
Step by step:

class Pet: → tells Python “I’m defining a new class named Pet”.
pass → means “nothing inside yet, just an empty shell so this is valid code”.

This is a class definition: the blueprint, not an actual pet yet.

Does this idea make sense so far: class Pet: pass is just the blueprint, and doesn’t create any real pet yet?

Yes, that makes sense

Great ✅

Next step: let’s actually make a pet from that blueprint.

In Python, we create an object (instance) by calling the class with parentheses:

Python
class Pet:
    pass # blueprint

luna = Pet() # create one Pet object
Step by step:

class Pet: + pass → defines the blueprint.
luna = Pet() → instantiates the class: it creates one actual object and stores it in the variable luna.

So Pet is the blueprint, luna is one pet object built from it.

Make sense that Pet is the class, and luna = Pet() is where the object is created?

Yes, that makes sense

Nice 👍

Now let’s see what we actually created.

If we run this:

Python
class Pet:
    pass # class definition (blueprint)

luna = Pet() # object / instance

print(type(luna))
Step by step what will happen:

Python reads the class definition.
luna = Pet() creates an object.
print(type(luna)) shows the type of luna — it should say something like <class '__main__.Pet'>.

You don’t need to run it yet — just think:

type(luna) should tell us that luna is a Pet.

Ready to try this in the code editor as your first custom class? 🐶

Yes, I’m ready

Awesome, let’s do it step by step 🧰

In the code editor, I want you to:

Type this exact code:
Python
```
class Pet:
    pass  # class definition (blueprint)

luna = Pet()  # create an object (instance)

print(type(luna))  # inspect the type of the object
```
Click Run first to check it works.
Then click Submit code so I can see it.

Don’t change anything yet — just get this working.

Once you’ve submitted, I’ll help you add information (attributes) to luna ✅

Python
```
class Pet:
    pass  # class definition (blueprint)

luna = Pet()  # create an object (instance)

print(type(luna))  # inspect the type of the object
```
Nice work, Ron — you just created and inspected your first custom object 🎉

You can see in the output:

Text

```
<class '__main__.Pet'>
That tells us:


luna is an object
Its type is the class Pet you defined
Now let’s add information to that object using an attribute.


Think of attributes as labeled facts about luna, like name and species.


In the editor, starting from your working code, add these two lines below luna = Pet():

Python
luna.name = "Luna"
luna.species = "dog"
And also add:

Python
print(luna.name)
print(luna.species)
So the full script should:


Define Pet
Create luna
Assign name and species to luna
Print them

Run it, then Submit code so I can check it 🐕

Python
class Pet:
    pass  # class definition (blueprint)

luna = Pet()  # create an object (instance)
luna.name = "Luna"
luna.species = "dog"
print(type(luna))  # inspect the type of the object
print(luna.name)
print(luna.species)
Beautifully done, Ron 🎯


```

You just did three key things:

Defined a class: the Pet blueprint
Instantiated it: luna = Pet() created an object
Added attributes after creation: luna.name, luna.species

And the output:

Text
<class '__main__.Pet'>
Luna
dog
confirms it all worked.

Quick check-in before we switch to a different example (like a Book):

Does it feel clear that:

class Pet: → blueprint
luna = Pet() → creates the object
luna.name = "Luna" → adds info to that object

Anything fuzzy in those three steps, or good to go to a new example?

That’s clear

Great, then let’s lock it in with a tiny new example 📘

We’ll switch from pets to books, but the pattern stays the same:

Define a class (blueprint) called Book
Create one object from it
Add 1–2 attributes after creation
Print the type and the attributes

In the code editor, write a script that does this:

class Book: with a pass inside (empty class)
Create one book object, for example my_book = Book()

Add attributes like title and author to my_book after it’s created
Use print(type(my_book))
Print the attributes (for example print(my_book.title))

Then:

Click Run
If it works, click Submit code so I can see it ✨
Python
```
class Book:
    pass  # class definition (blueprintt

my_book = Book()  # create an object (instance)
my_book.title = "The Hobbit"
my_book.author = "J.R.R. Tolkein"
print(type(my_book))  # inspect the type of the object
print(my_book.title)
print(my_book.author)
```
Take your time with the question below 👇

You have unlocked your first custom object! 🎉 Which line creates a new object from the Pet class?

A.
```
class Pet:
```


B.
```
luna = Pet()
```


C.
```
luna.name = "Luna"
```


D.
```
type(luna)
```

Nice job, Ron — you nailed it 🎯

luna = Pet() is exactly the line that creates a new object (instance) from the Pet class.

Quick recap of each line’s role:

class Pet: → defines the class (the blueprint)
luna = Pet() → instantiates the class → makes one actual Pet object
luna.name = "Luna" → adds an attribute to that object
type(luna) → lets us inspect what kind of object luna is

You’ve shown you can:

Define a minimal class ✅
Create objects and inspect their type ✅
Add attributes after instantiation ✅
Tell class definition and instantiation apart ✅

You’ve basically unlocked the door to real object-oriented Python 🗝️

Are you ready to wrap up this lesson and move on to the next one, or do you want one more quick practice first?

Move on to the next lesson

Love it. You’re ready 🚀

Quick snapshot of what you accomplished in this lesson:

You defined minimal classes with class Pet: and class Book:
You created objects like luna = Pet() and my_book = Book()
You assigned attributes after creation (luna.name, my_book.title, etc.)
You used type(...) to confirm what kind of objects you made
You clearly distinguished:
Class definition → blueprint
Instantiation → variable = Class() makes a real object

This is the foundation of object-oriented Python. From here, new ideas (like initializers and methods) will build on exactly what you just did.

```
You can hit Next Lesson when you’re ready, and I’ll walk with you through the next step 👣


class Pet:
    pass  # class definition (blueprint)

luna = Pet()  # create an object (instance)

print(type(luna))  # inspect the type of the object


class Pet:
    pass  # class definition (blueprint)

luna = Pet()  # create an object (instance)
luna.name = "Luna"
luna.species = "dog"
print(type(luna))  # inspect the type of the object
print(luna.name)
print(luna.species)

class Book:
    pass  # class definition (blueprintt

my_book = Book()  # create an object (instance)
my_book.title = "The Hobbit"
my_book.author = "J.R.R. Tolkein"
print(type(my_book))  # inspect the type of the object
print(my_book.title)
print(my_book.author)
x
```